

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS:

Total Marks: 25

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.
4. An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5. A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

*I Mathusri P-192311363(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Mathusri P

RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

Q1. REST vs SOAP for Mobile and Web Application

Problem Understanding

The company needs a web service that can be accessed by both mobile and web applications. The service should provide fast communication, support many users, be scalable, and maintain adequate security. Two suitable approaches are **REST** and **SOAP**.

Solution 1 – REST

REST (Representational State Transfer) is an architectural style commonly used for web APIs. It generally uses HTTP methods such as GET, POST, PUT and DELETE. Data is commonly exchanged using lightweight JSON.

REST is simple, lightweight and suitable for mobile applications because JSON responses require less data than traditional XML-based SOAP messages. REST services are also generally easier to develop and scale using standard HTTP infrastructure.

However, REST does not provide the same built-in enterprise messaging features as SOAP. Advanced requirements such as formal contracts and certain WS-* specifications may require additional technologies.

Solution 2 – SOAP

SOAP (Simple Object Access Protocol) is an XML-based messaging protocol. It provides a formal communication structure using SOAP Envelope, Header and Body. SOAP can support enterprise-level features such as structured messaging, security extensions and reliable messaging mechanisms.

SOAP is suitable when the application requires strict contracts, standardized XML messages, and complex enterprise integration. However, SOAP messages are comparatively verbose, resulting in greater processing and network overhead.

Comparison

REST is lightweight and generally provides better performance for mobile and web applications because JSON messages are smaller and HTTP infrastructure can be used directly. SOAP provides stronger formalization and enterprise-level capabilities but introduces additional XML processing overhead.

For scalability, REST works well with stateless server design and can be deployed behind load balancers. SOAP can also be scaled, but its heavier message processing can increase resource consumption.

For security, REST can use HTTPS, authentication tokens, OAuth 2.0 and other security mechanisms. SOAP can use HTTPS as well as WS-Security for message-level security.

Selected Solution: REST

REST is selected for this application because the main clients are mobile and web applications. REST provides a lightweight communication mechanism, easier development, lower bandwidth consumption and good scalability.

The proposed architecture is:

Mobile App / Web App → REST API → Application Server → Database

The API can use JSON for request and response data. HTTPS should be used to protect communication, while authentication tokens can be used to control access.

Feasibility

The REST solution is practical because it can support different client platforms using the same API. Stateless requests allow additional servers to be added when traffic increases. Caching and load balancing can further improve performance.

Conclusion

Although SOAP is appropriate for complex enterprise systems requiring strict contracts and specialized messaging features, **REST is the better choice for a mobile and web application** because of its simplicity, performance, scalability and efficient data exchange.

Q2. Scalable Service Architecture for High User Traffic

Problem Understanding

The online platform is expected to receive a large number of users simultaneously. Therefore, the system must handle high traffic without becoming slow or unavailable. The architecture should provide **scalability, reliability, efficient data processing, and easy maintenance**.

The basic system consists of a client, server, API and database. However, a single server and database can become a bottleneck when the number of users increases.

Solution 1 – Traditional Architecture

In the traditional architecture, all users communicate with a single application server.

Client → Server → API → Database

The client sends a request to the server. The server processes the request through the API and communicates with the database. The database returns the required information, which is then sent back to the client.

This design is simple and easy to implement. It is suitable for small applications with limited traffic. However, when thousands of users access the system simultaneously, the single server may become overloaded. Database access can also become a bottleneck.

Solution 2 – Scalable Architecture

A better approach is to use multiple application servers behind a **load balancer**.

Users → Load Balancer → Multiple API Servers → Database

The load balancer distributes incoming requests among multiple servers. If one server becomes heavily loaded, requests can be directed to other available servers.

The API servers should preferably be **stateless**, meaning that important session information is not stored only on one server. This allows any available server to process a request.

A caching layer can also be introduced to store frequently requested information. This reduces repeated database operations and improves response time.

Alternative Improved Design

A more scalable architecture can be designed as:

Client → Load Balancer → API Gateway → Application Servers → Cache → Database

The **API Gateway** acts as a central entry point for client requests. It can perform authentication, request routing, rate limiting and monitoring.

Multiple application servers process requests in parallel. A cache stores frequently accessed data, while the database remains responsible for permanent storage.

For very high traffic, the database can also use **read replicas** so that read operations are distributed instead of sending every request to one database server.

Evaluation of Solutions

The single-server architecture is easier to develop and maintain, but its scalability is limited. If the server fails, the complete service may become unavailable.

The multiple-server architecture provides better scalability and reliability because workload can be distributed across several servers. However, it requires additional infrastructure and proper monitoring.

The API Gateway and caching approach provides even greater scalability. Caching reduces database load, while load balancing distributes requests across servers. However, the architecture is more complex and requires careful configuration.

Selected Solution

The **scalable architecture using a load balancer, multiple API servers, caching, and a database** is selected.

The proposed architecture is:

Client → Load Balancer → API Gateway → Multiple Application Servers → Cache/Database

This design can support increasing numbers of users by adding more application servers when required.

Justification

The selected architecture improves **performance** because requests are processed by multiple servers.

It improves **scalability** because additional servers can be added without changing the client application.

It improves **availability** because failure of one application server does not necessarily stop the entire service.

Caching reduces repeated database requests and improves response time for frequently accessed information.

Database read replicas can further improve performance when the platform has a large number of read operations.

Feasibility

This architecture is practical for an online platform because its components can be deployed incrementally. Initially, the system can use a small number of servers and later increase capacity according to traffic.

Monitoring tools can track CPU usage, memory usage, response time and request volume. Automatic scaling can also be used to add or remove application servers according to demand.

Conclusion

A single-server architecture is suitable for small applications but becomes inefficient under heavy traffic. A **load-balanced, multi-server architecture with an API gateway, caching and scalable database design** provides better performance, reliability and scalability.

Therefore, the proposed architecture is the most suitable solution for an online platform expecting a large number of simultaneous users.

Q3. Secure Communication Protocol for Banking Application

Problem Understanding

A banking application handles highly sensitive information such as customer details, account information, login credentials and financial transactions. Therefore, communication between the client and server must provide **strong security, confidentiality, data integrity, authentication and reliable transmission**.

The communication protocol must prevent attackers from reading or modifying sensitive banking information.

Solution 1 – HTTP

HTTP can be used for communication between a client and server, but the data is transmitted without encryption.

For a banking application, normal HTTP is **not suitable** because usernames, passwords and transaction information could be exposed during transmission.

Solution 2 – HTTPS

HTTPS is HTTP combined with **TLS (Transport Layer Security)**. It encrypts communication between the client and server.

HTTPS provides:

- **Confidentiality** – prevents unauthorized users from reading transmitted data.
- **Integrity** – helps detect modification of data during transmission.
- **Authentication** – TLS certificates help the client verify the server.
- **Secure communication** – protects sensitive banking transactions.

Therefore, HTTPS is much more suitable for banking applications.

Solution 3 – HTTP/2 or HTTP/3 over TLS

Modern HTTP versions can also operate securely over TLS. They can improve communication efficiency through features such as multiplexing and improved connection handling.

However, they still depend on TLS for protecting sensitive information.

Comparison

HTTP provides basic communication but does not encrypt transmitted information, making it unsuitable for banking.

HTTPS provides encrypted and authenticated communication and is widely suitable for secure web applications.

HTTP/2 or HTTP/3 with TLS can provide additional performance benefits while maintaining secure communication. However, the fundamental security requirement remains the use of TLS.

Selected Solution: HTTPS with TLS

HTTPS with TLS is selected as the primary communication protocol for the banking application.

The communication architecture can be represented as:

Banking Client → HTTPS/TLS → Banking Server → Secure Backend Services → Database

When a customer performs an operation, the request is encrypted before being transmitted to the banking server. The server verifies the secure connection, processes the request and sends an encrypted response.

Security Justification

HTTPS protects customer information from network attackers by encrypting communication. TLS also helps ensure that data has not been modified while travelling between the client and server. Server certificates provide authentication, reducing the risk of communicating with an impersonated server.

Additional security mechanisms such as **multi-factor authentication, secure session management, access control and transaction authorization** should also be implemented.

Reliability and Performance

HTTPS provides reliable communication while protecting data. TLS introduces some processing overhead because encryption and decryption are required, but modern systems are designed to handle this efficiently.

Using HTTP/2 or HTTP/3 over TLS can further improve performance by allowing more efficient communication between clients and servers.

For a banking system, **security should have higher priority than a small reduction in communication overhead** because financial data is extremely sensitive.

Alternative Approach

An alternative is to use **mutual TLS (mTLS)** for selected internal banking services. In normal TLS, the client verifies the server. In mutual TLS, both the client and server authenticate each other using certificates.

This provides stronger authentication for communication between trusted internal services, although it requires additional certificate management.

Feasibility

HTTPS is highly practical because modern browsers and mobile applications support it directly. Banking applications can deploy TLS certificates on their servers and enforce secure HTTPS connections.

The system should also disable insecure HTTP access and redirect or reject unencrypted requests.

Conclusion

For a banking application, **HTTPS with TLS is the most suitable communication protocol** because it provides confidentiality, integrity and server authentication while maintaining reliable communication.

Therefore, the recommended approach is:

Client → HTTPS/TLS → Secure Banking Server → Protected Backend

For highly sensitive internal service-to-service communication, **mutual TLS** can be added as an additional security measure.

Q4. API Integration Strategy for Third-Party Services

Problem Understanding

The application needs to communicate with third-party services such as **payment gateways, map services, notification services, or other external APIs**. The integration must securely send requests, authenticate with the external service, process responses, and handle failures.

The main objectives are **security, reliability, efficient communication, error handling and smooth user experience**.

Solution 1 – Direct API Integration

In direct integration, the application communicates directly with the third-party API.

Application → Authentication → Third-Party API → Response → Application

For example, when a user makes a payment, the application sends the required transaction information to the payment gateway. The gateway processes the request and returns the payment status.

Similarly, for a map service, the application sends location information and receives map or location data.

This approach is simple and suitable when only a few external services are required.

Solution 2 – API Gateway / Integration Layer

A better approach for a larger application is to introduce an **API Gateway or integration layer** between the application and third-party services.

Application → API Gateway → Third-Party Service → API Gateway → Application

The gateway manages authentication, request routing, rate limiting, logging and error handling. This prevents the main application from becoming tightly dependent on individual third-party services.

API Request Handling

The integration process can be performed through the following steps:

User Request → Application Validation → Authentication → API Request → Third-Party Service → Response Validation → Process Response → Update Application → Display Result

For a payment service, the application first validates the transaction details. It then sends an authenticated request to the payment gateway using HTTPS.

The external service processes the request and returns a response such as successful, failed or pending.

The application validates the response before updating the transaction status.

Authentication

Third-party APIs require authentication to ensure that only authorized applications can access them.

Common approaches include:

- API keys
- OAuth 2.0 access tokens
- Signed requests
- Client credentials
- Mutual TLS for highly secure systems

Sensitive API keys and credentials should not be directly exposed to users or stored in client-side code.

Response Processing

The application should validate the response received from the third-party service.

For example:

Payment Request → Payment Gateway → Success/Failure Response → Validate Response → Update Payment Status

If the response indicates success, the application can update the transaction. If the response indicates failure, an appropriate message should be shown to the user.

Error Handling

Third-party services may experience network failures, timeout errors, invalid responses or temporary unavailability.

The application should therefore implement:

- Request timeouts
- Retry mechanisms
- Proper error messages
- Logging
- Response validation
- Rate-limit handling
- Fallback mechanisms

Retries should be carefully controlled, especially for payment operations, to avoid accidentally performing the same transaction more than once.

Alternative Approach

An alternative is to use an **integration service or message queue**.

Instead of waiting for every third-party operation to complete immediately:

Application → Message Queue → Integration Service → Third-Party API

This approach is useful for operations that do not require an immediate response. It allows failed requests to be retried later and prevents temporary third-party failures from directly affecting the main application.

For example, sending notifications can be performed asynchronously using a queue.

Selected Solution

For a system integrating several third-party services, the **API Gateway with a dedicated integration layer** is the preferred solution.

It provides centralized authentication, request management, monitoring, rate limiting and error handling.

For long-running or non-critical operations, a **message queue** can be added to improve reliability.

Feasibility

The proposed design is practical because external services can be added or replaced without significantly changing the main application.

The integration layer also makes the system easier to monitor and maintain. Secure HTTPS communication and protected credentials help prevent unauthorized access.

Conclusion

A reliable API integration strategy should securely authenticate requests, validate responses and handle failures properly. Direct API integration is suitable for simple applications, while an **API Gateway and integration layer** is more suitable for complex systems with multiple external services.

Therefore, the recommended architecture is:

Application → API Gateway → Integration Layer → Third-Party Services

with **message queues and retry mechanisms** used where asynchronous processing and improved reliability are required.

Q5. Error Handling and Fault Tolerance Mechanism

Problem Understanding

A web service may frequently encounter errors during client requests because of invalid input, network failure, server overload, database problems, timeout, or temporary unavailability of another service. If these errors are not handled properly, users may receive incorrect results or the entire application may become unavailable.

The objective is to design a mechanism that provides **error detection, proper error handling, fault tolerance, reliability, and a smooth user experience.**

Solution 1 – Basic Error Handling

The first approach is to validate every client request before processing it.

Client Request → Validate Request → Process Request → Generate Response

If the request contains invalid information, the service returns an appropriate error message instead of processing the request.

For example, if a required user ID is missing, the service should return:

“Invalid request: User ID is required.”

This prevents unnecessary processing and reduces errors.

Solution 2 – Retry and Timeout Mechanism

Sometimes an error occurs because of a temporary network or service failure. In such cases, the system can retry the request after a short interval.

Request → Service Failure → Check Retry Condition → Retry → Success / Failure Response

A timeout should also be configured so that the application does not wait indefinitely for an unavailable service.

However, retries should be limited because continuous retries can increase server load.

Solution 3 – Fault-Tolerant Architecture

A more advanced approach is to use multiple application servers with load balancing.

Client → Load Balancer → Server 1 / Server 2 / Server 3 → Database

If one server fails, the load balancer can send requests to another available server.

This improves availability and prevents a single server failure from stopping the entire application.

Error Classification

Errors can be classified into different categories.

Client Errors:

These occur because of invalid input or incorrect requests. For example, missing fields or invalid authentication.

Server Errors:

These occur because of problems within the application server, database or backend service.

Network Errors:

These occur due to connection failure, timeout or temporary network problems.

External Service Errors:

These occur when a third-party service becomes unavailable or returns an unexpected response.

Each type of error should be handled differently.

Error Handling Mechanism

The proposed mechanism is:

Client Request → Input Validation → Authentication → Process Request → Check Service/Database → Generate Response

If an error occurs:

Error → Identify Error Type → Log Error → Retry if Suitable → Return Safe Error Response

Sensitive technical information such as database passwords or internal server details should never be displayed to users.

Instead, the user should receive a simple message such as:

“Service temporarily unavailable. Please try again later.”

Fault Tolerance Techniques

Several techniques can improve reliability.

1. Retry Mechanism:

Temporary failures can be handled by retrying the request a limited number of times.

2. Timeout:

A request should stop waiting after a predefined period.

3. Circuit Breaker:

If an external service repeatedly fails, the circuit breaker temporarily stops sending requests to it. This prevents continuous failures and allows the service to recover.

4. Load Balancing:

Requests can be distributed among multiple servers.

5. Backup and Recovery:

Important data should be backed up regularly so that it can be recovered after system failure.

6. Logging and Monitoring:

Errors should be recorded and monitored to identify recurring problems.

Alternative Strategy

An alternative approach is to use an **asynchronous message queue**.

Client → API → Message Queue → Service → Response/Status

If the backend service is temporarily unavailable, the request can remain in the queue and be processed later.

This is especially useful for non-immediate operations such as notifications, report generation and background processing.

Selected Solution

The recommended solution is a **combination of validation, timeout, limited retry, circuit breaker, load balancing, logging and monitoring**.

For asynchronous tasks, a message queue can also be added.

This combination provides better reliability than depending on a single error-handling technique.

Feasibility

The proposed mechanism is practical for modern web services because individual components can be introduced gradually.

Initially, the application can implement input validation, proper error responses and logging.

As traffic and system complexity increase, load balancing, circuit breakers, monitoring and message queues can be introduced.

Conclusion

A web service should not simply display an error when a request fails. It should detect the type of failure, handle it safely and recover whenever possible.

The recommended architecture is:

Client → API → Validation → Service → Database/External Service → Response
with:

Error → Log → Retry/Timeout → Circuit Breaker/Fallback → User-Friendly Response

Therefore, combining **error handling and fault-tolerance techniques** provides a reliable service, reduces downtime and ensures a better user experience even when failures occur.
